

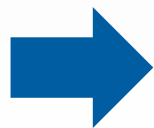
算法效率分析基础

讲解人：陆伟

September 2, 2025

目录

CONTENTS



1

算法效率的度量

2

函数渐进的界

3

算法复杂性分析基本方法

4

递归与非递归算法比较

5

经验与实验分析方法

算法效率的度量

- 算法效率的高低体现在运行该算法所需要耗费资源的多少，对于计算机来讲，最重要的资源是时间和空间，因此，算法效率又可分为时间效率和空间效率。
- 分别用 N ， I 和 A 表示要解决问题的规模、算法的输入和算法本身，用 C 表示复杂性，那么，应该有 $C = F(N, I, A)$ 。如果把时间复杂性与空间复杂性分开，分别用 T 和 S 表示，则 $T = F(N, I, A)$ ， $S = F(N, I, A)$ 。
- $T = T(N, I)$ ， $S = S(N, I)$
- 根据 $T = T(N, I)$ 的概念，它应该是算法在一台“抽象的计算机”上运行所需要的时间。

为何要抽象？讨论

算法效率的度量

- 设该“抽象的计算机”所提供的元运算有 k 种，分别记为 $O_1, O_2, \dots, O_k$ ，又设每执行一次这些元运算所耗费的时间分别为 $t_1, t_2, \dots, t_k$ 。对于给定算法 A ，统计其执行过程中用到的元运算 O_i 的次数，记为 e_i ， $i=1, 2, \dots, k$ 。 $e_i = e_i(N, I)$ 。

$$T(N, I) = \sum_{i=1}^k t_i e_i(N, I) \quad T(N, I) = \sum_{i=1}^k t_i e_i(N, I) = e^*(N, I)$$

- 其中， t_i 是与 N 和 I 无关的常数。

e_i 为 N 和 I 的函数， I 如何表示和确定？

算法效率的度量

□ 我们不可能对规模为 N 的每一种合法输入 I 都去统计 $e_i(N, I)$, $i=1,2,\dots,k$ 。

$$T_{\max}(N) = \max_{I \in D_N} T(N, I) = \max_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, I^*)$$

$$T_{\min}(N) = \min_{I \in D_N} T(N, I) = \min_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, \tilde{I})$$

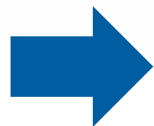
$$T_{\text{avg}}(N) = \sum_{I \in D_N} P(I) T(N, I) = \sum_{I \in D_N} P(I) \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, I)$$

□ 关于摊销效率

算法效率最终抽象为问题规模的函数，针对同一问题的不同算法，如何进行效率比较？

目录

CONTENTS



1

算法效率的度量

2

函数渐进的界

3

算法复杂性分析基本方法

4

递归与非递归算法比较

5

经验与实验分析方法

函数渐进的界

□ 设 f 和 g 是定义域为自然数集 $\mathbb{N}$ 上的函数

(1) $f(n) = O(g(n))$

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$

(2) $f(n) = \Omega(g(n))$

若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$

(3) $f(n) = o(g(n))$

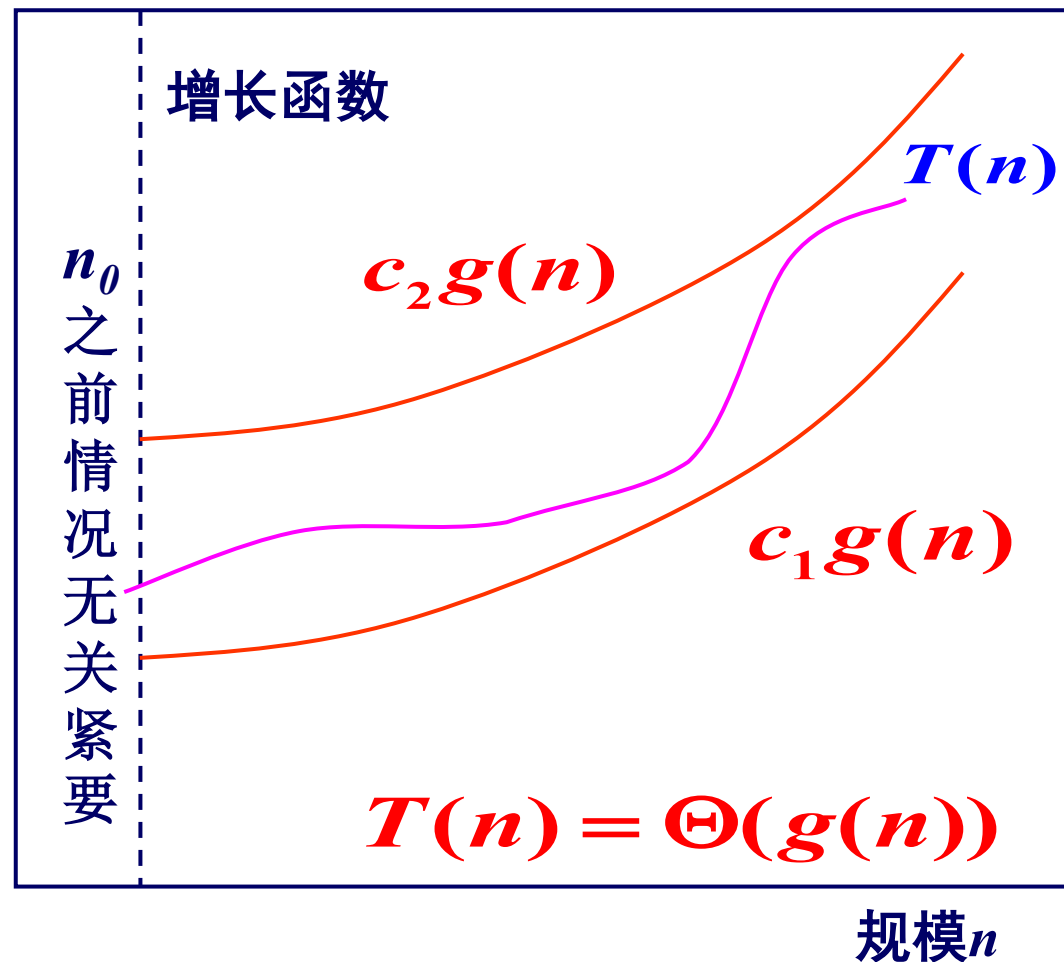
对任意正数 c 存在 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) < cg(n)$

(4) $f(n) = \omega(g(n))$

对任意正数 c 存在 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) < f(n)$

(5) $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$

(6) $O(1)$ 表示常数函数



函数渐进的界

□ 函数渐进界的基本性质 (1)

(1) 若 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, c 为大于0的常数, 那么 $f(n) = \Theta(g(n))$

(2) 若 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, c 为大于0的常数, 那么 $f(n) = o(g(n))$

(3) 若 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, c 为大于0的常数, 那么 $f(n) = \omega(g(n))$

函数渐进的界

□ 函数渐进界的基本性质 (2)

(1) 如果 $f=O(g)$ 且 $g=O(h)$, 那么 $f=O(h)$.

(2) 如果 $f=\Omega(g)$ 且 $g=\Omega(h)$, 那么 $f=\Omega(h)$.

(3) 如果 $f=\Theta(g)$ 和 $g=\Theta(h)$, 那么 $f=\Theta(h)$.

(4) $O(f(n))+O(g(n)) = O(\max\{f(n),g(n)\})$

(5) $O(f(n))+O(g(n)) = O(f(n)+g(n))$

(6) $O(f(n))*O(g(n)) = O(f(n)*g(n))$

函数渐进的界

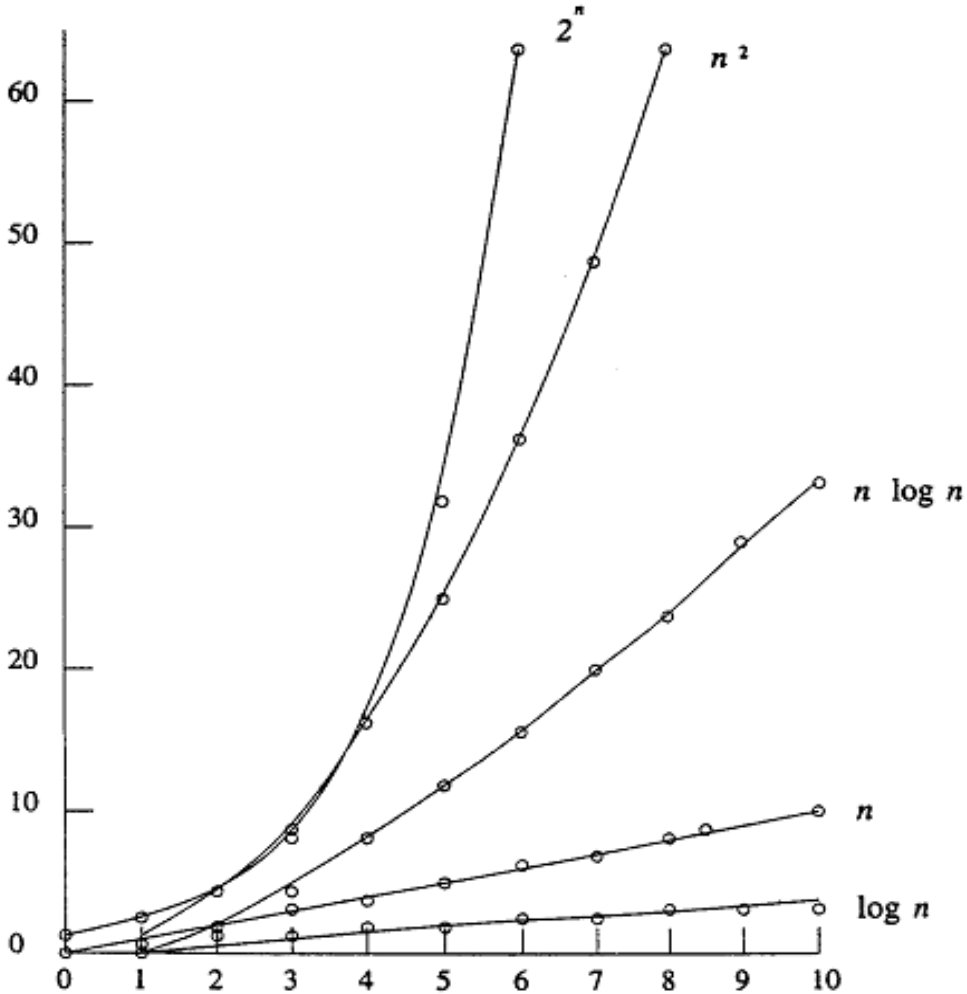
例

- 多项式函数 $f(n)=a_0+a_1n+a_2n^2+\dots+a_dn^d$, 其中 $a_d \neq 0$, 证明 $f(n) = \Theta(n^d)$
- 证明 $\log_2 n = o(\sqrt{n})$
- 证明 $\log_a n = \Theta(\log_b n)$
- 对于 $b > 1$ 和 $\alpha > 0$, $\log_b n = o(n^\alpha)$, $n^\alpha = o(b^n)$
- $n! = o(n^n)$, $n! = \omega(2^n)$, $\log(n!) = \Theta(n \log n)$

函数渐进的界

算法基本渐进效率类型

类 型	名 称
1	常量
logn	对数
n	线性
nlogn	n-logn-n
n ²	二次
n ³	三次
2 ⁿ	指数
n!	阶乘



列子。
的执行时间也会趋于无穷大。
因子。

均效率属于此类型。
方阵的某些特定算法。

这个术语常用在更广泛的
长速度更快的算法如阶乘。

目录

CONTENTS



1

算法效率的度量

2

函数渐进的界

3

算法复杂性分析基本方法

4

递归与非递归算法比较

5

经验与实验分析方法

算法复杂性分析基本方法

□ 算法复杂性分析基本步骤

$$T(N, I) = \sum_{i=1}^k t_i e_i(N, I) = e^*(N, I)$$

- (1) 确定表示输入规模的参数。
- (2) 找出算法的基本操作。
- (3) 检查基本操作的执行次数是否只依赖于输入规模。这决定是否需要考虑最差、平均以及最优情况下的复杂性。
- (4) 对于非递归算法，建立算法基本操作执行次数的求和表达式；对于递归算法，建立算法基本操作执行次数的递推关系及其初始条件。。
- (5) 利用求和公式和法则建立一个操作次数的闭合公式，或者求解递推关系式，确定增长的阶。

算法复杂性分析基本方法

□ 非递归算法复杂性分析常见求和公式

□ 等差数列 $\sum_{k=1}^n a_k$

□ 等比数列 $\sum_{k=0}^n aq^k$

□ 调和级数 $\sum_{k=1}^n \frac{1}{k}$

□ 对数求和 $\sum_{i=1}^n \log i$

算法复杂性分析基本方法

□ 非递归算法复杂性分析

例

算法 **maxElement**(A[0..*n*-1])

//求给定数组中的最大元素

//输入: 实数数组A[0..*n*-1]

//输出: A中的最大元素

maxval ← A[0]

for *i* ← 1 to *n*-1 do

 if A[*i*] > *maxval*

maxval ← A[*i*]

return *maxval*

- (1) 算法输入规模: 可以用数组元素个数*n*度量
- (2) 基本操作: 比较与赋值两种, 选择比较
- (3) 比较操作只与输入规模相关, 不用考虑最坏、平均、最好情况

(4) 建立基本操作执行次数求和表达式

$$T(n) = \sum_{i=1}^{n-1} 1 = n - 1$$

(5) 确定增长的阶 $T(n) = O(n)$

算法复杂性分析基本方法

□ 非递归算法复杂性分析

例

算法 uniqueElements(A[0..n-1])

//验证给定数组中元素是否全部唯一

//输入：实数数组A[0..n-1]

//输出：如果A中的元素全部唯一，
返回“true”，否则，返回“false”

for $i \leftarrow 0$ to $n-2$ do

 for $j \leftarrow i+1$ to $n-1$ do

 if $A[i] = A[j]$ return false

return true

(1) 算法输入规模：可以用数组元素个数 n 度量

(2) 基本操作：比较

(3) 比较操作执行次数与输入相关，需要考虑
最坏、平均、最好情况

(4) 建立基本操作执行次数求和表达式

最好： $O(1)$

最坏：

$$\begin{aligned}\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 &= \sum_{i=0}^{n-2} (n-1) - (i+1) + 1 = i = \sum_{i=0}^{n-2} n - i - 1 \\ &= 1 + 2 + \dots + (n-1) = n(n-1)/2\end{aligned}$$

(5) 确定增长的阶 $O(n^2)$

平均效率如何分析？

算法复杂性分析基本方法

例

□ 非递归算法复杂性分析

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} (t_i - 1) + c_6 \sum_{i=1}^{n-1} (t_i - 1) + c_7(n-1)$$

最好情况下, $t_i=1$, for $1 \leq i < n$

$$T_{\min}(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) \\ = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7) = O(n)$$

最坏情况下, $t_i \leq i+1$, for $1 \leq i < n$

$$\sum_{i=1}^{n-1} (i+1) = \frac{n(n+1)}{2} - 1 \quad \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$
$$T_{\max}(n) \leq c_1 n + c_2(n-1) + c_3(n-1) + c_4 \left(\frac{n(n+1)}{2} - 1 \right) + c_5 \left(\frac{n(n-1)}{2} \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7(n-1)$$
$$= \frac{c_4 + c_5 + c_6}{2} n^2 + \left(c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7 \right) n - (c_2 + c_3 + c_4 + c_7)$$
$$= O(n^2)$$

```
template<class Type>
void insertion_sort(Type *a, int n)
{
    Type key;                                // cost times
    for (int i = 1; i < n; i++){              // c1    n
        key=a[i];                             // c2    n-1
        int j=i-1;                             // c3    n-1
        while( j>=0 && a[j]>key ){             // c4    sum of t_i
            a[j+1]=a[j];                       // c5    sum of (t_i-1)
            j--;                                // c6    sum of (t_i-1)
            a[j+1]=key;                         // c7    n-1
        }
    }
```

算法复杂性分析基本方法

□ 递归算法复杂性分析

递归算法时间复杂度典型递推方程之一

$$T(n) = \begin{cases} O(1) & n=1 \\ aT(n-1)+f(n) & n>1 \end{cases}$$

讨论与
理解

$$T(n) = a^{n-1} T(1) + \sum_{i=2}^n a^{n-i} f(i)$$

推导

(1) 若取 $a=2$, $f(n)=O(1)$, 汉诺塔问题

$$T(n) = O(2^n - 1)$$

(2) 若取 $a=1$, $f(n)=n-1$, 插入排序最坏情况

$$T(n) = O(n^2)$$

例

算法复杂性分析基本方法

□ 递归算法复杂性分析

递归算法时间复杂度典型递推方程之二

$$T(n) = \begin{cases} O(1) & n=1 \\ aT(\frac{n}{b}) + f(n) & n>1 \end{cases}$$

讨论与
理解

$$T(n) = n^{\log_b a} T(1) + \sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j})$$

推导

(1) 若 $f(n)=c$

实例

$$T(n) = \begin{cases} O(n^{\log_b a}) & a \neq 1 \\ O(\log n) & a = 1 \end{cases}$$

(2) 若 $f(n)=cn$

实例

$$T(n) = \begin{cases} O(n) & a < b \\ O(n \log n) & a = b \\ O(n^{\log_b a}) & a > b \end{cases}$$

算法复杂性分析基本方法

□ 递归算法复杂性分析

```
void hanoi(int n, int a, int b, int c) {  
    if (n = 1) move(a, c);  
    else {  
        hanoi(n-1, a, c, b);  
        move(a, c);  
        hanoi(n-1, b, a, c);  
    }  
}
```

求解汉诺塔
问题算法

例

$$T(n) = \begin{cases} O(1) & n=1 \\ 2T(n-1)+1 & n>1 \end{cases}$$

$$T(n)=O(2^n-1)$$

算法复杂性分析基本方法

□ 递归算法复杂性分析 $T(1)=O(1)$

$$(1) T(n)=T(n/2)+O(1)$$

二分查找算法递推方程

$$(2) T(n)=2T(n/2)+(n-1)$$

归并排序算法递推方程

$$(3) T(n)=T(n-1)+O(1)$$

插入排序最好情况

$$(4) T(n)=T(n-1)+(n-1)$$

插入排序最坏情况

$$(5) T(n)=T(n/3)+T(2n/3)+n$$

练习

算法复杂性分析基本方法

```
for i←0 to n-1 do // 行循环
  for j←0 to n-1 do // 列循环
    M[i][j]←0 // 初始化
    for k←0 to n-1 do // 用变量 k 表示变化的脚标
      M[i][j]←M[i][j] + A[i][k] * B[k][j]
return M
```

方阵乘法

练习

```
int binary(int n)
  count←1
  while n > 1 do
    count←count + 1
    n←n%2
  return count
```

十进制数的
二进制位数

目录

CONTENTS



1

算法效率的度量

2

函数渐进的界

3

算法复杂性分析基本方法

4

递归与非递归算法比较

5

经验与实验分析方法

递归与非递归算法比较

关于递归 Recursive

□ 阶乘问题

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

递归实现

```
int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

非递归实现

```
int factorial(int n){  
    int fn=1;  
    for(int i=2; i<=n; i++)  
        fn=fn*i;  
    return fn;  
}
```

参照代码 MyComputer.java (factorial部分), 执行并分析效率

课堂演示

递归与非递归算法比较

□ 斐波那契数列

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

递归实现

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

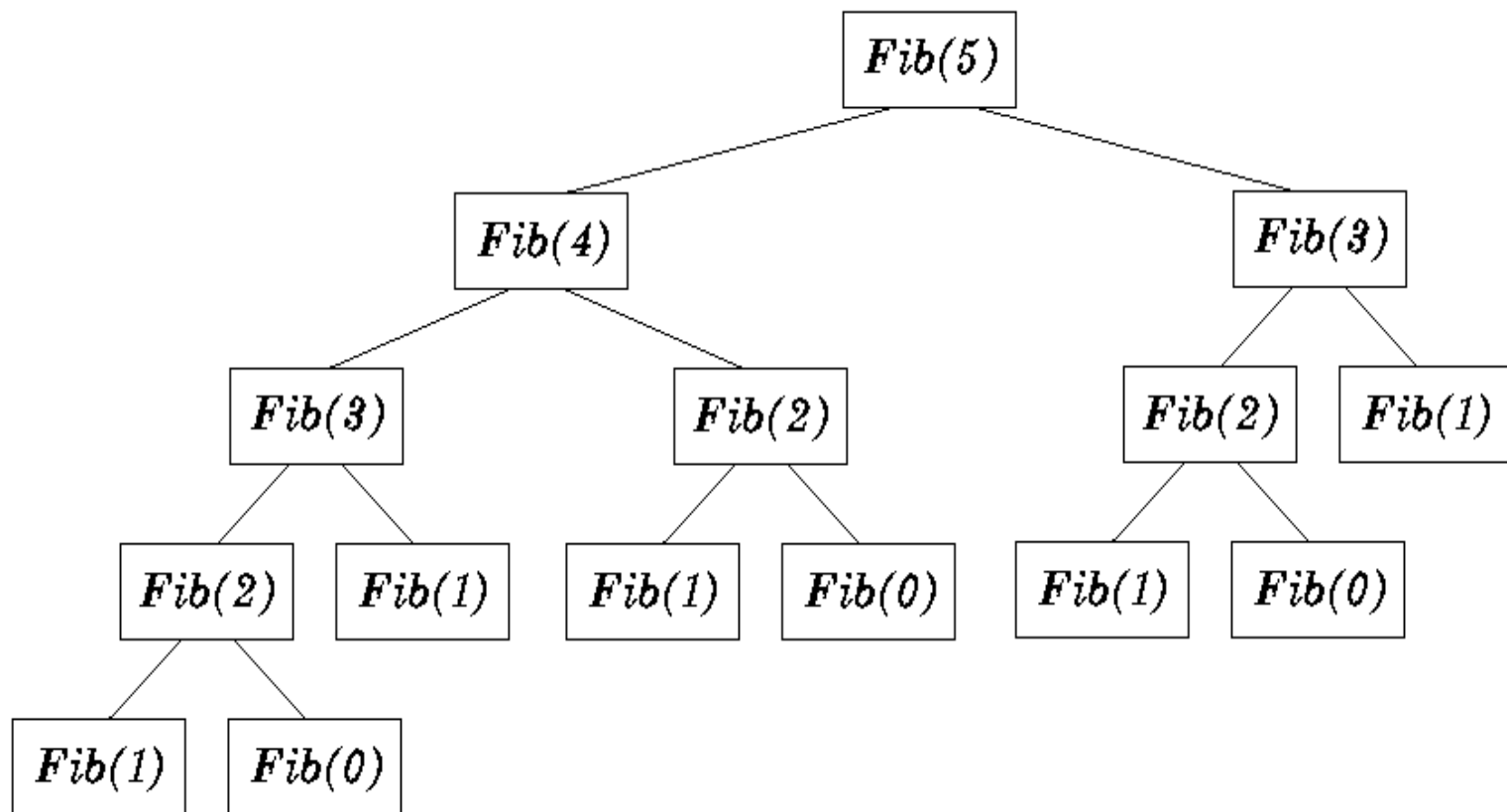
非递归实现

```
int fibonacci(int n){  
    int[] a = new int[2];  
    a[0] = 1; a[1] = 1;  
    for (int i = 2; i < n; i++) {  
        a[i%2] = a[(i-1)%2] + a[(i-2)%2];  
    }  
    return a[n % 2];  
}
```

参照代码 MyComputer.java (fibonacci部分) , 执行并分析效率

递归与非递归算法比较

斐波那契数列递归求解效率低下的重要原因在于大量重复计算



递归与非递归算法比较

□ 关于函数的递归与非递归定义问题

$$n! = n * (n-1)!$$



$$n! = n * (n-1) * \dots * 1$$

$$F(n) = F(n-1) + F(n-2)$$



?

$$F(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n = \frac{1}{\sqrt{5}} (\phi^n - \hat{\phi}^n)$$

$$\phi = (1 + \sqrt{5}) / 2 \approx 1.61803 \quad \hat{\phi} = -1 / \phi \approx -0.61803$$

并非所有递归算法都有非递归定义。

Ackerman函数

$$\left\{ \begin{array}{ll} A(1,0) = 2 \\ A(0,m) = 1 & m \geq 0 \\ A(n,0) = n + 2 & n \geq 2 \\ A(n,m) = A(A(n-1,m), m-1) & n, m \geq 1 \end{array} \right.$$

目录

CONTENTS



1

算法效率的度量

2

函数渐进的界

3

算法复杂性分析基本方法

4

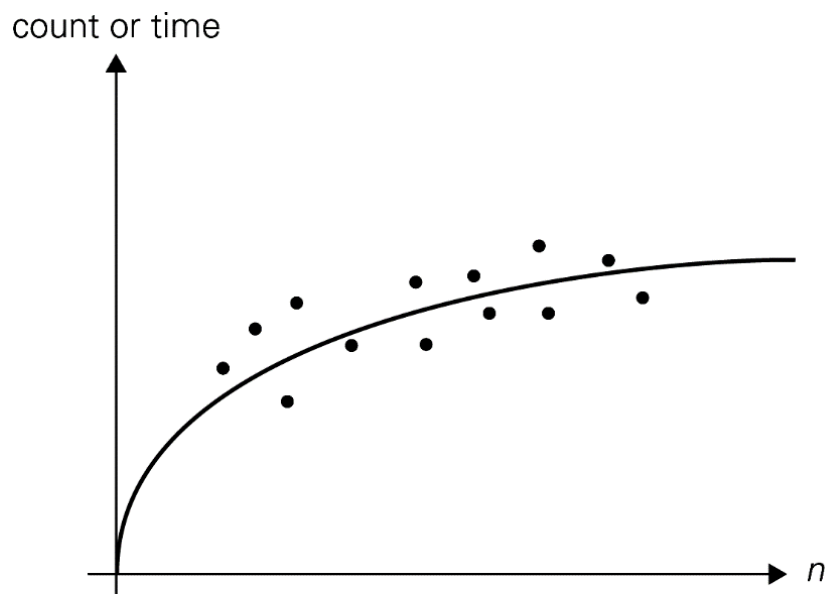
递归与非递归算法比较

5

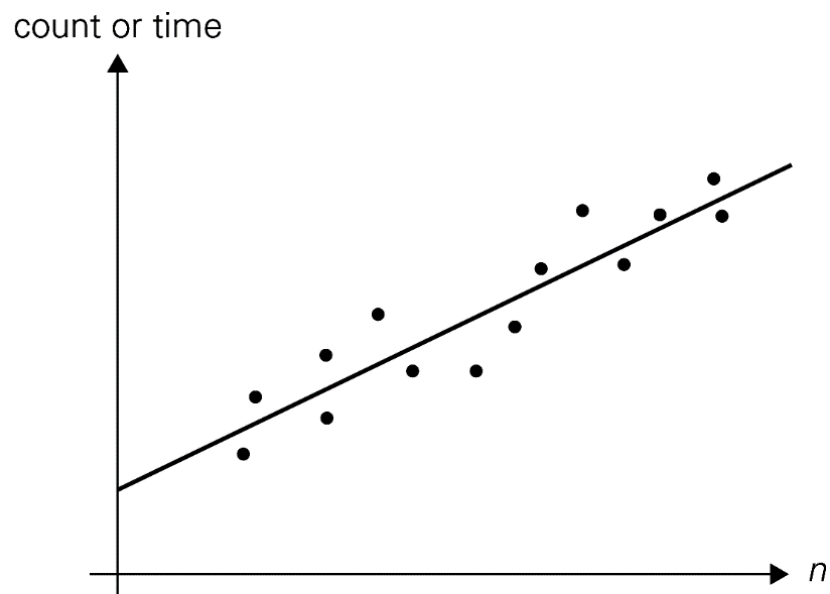
经验与实验分析方法

经验与实验分析法

- ❑ 数学远远不是万能的，即使许多貌似简单的算法，有时也很难用数学的精确性和严格性来分析，尤其对一些综合性算法，或是在做平均效率分析的时候。
- ❑ 除了可以对算法的效率做数学分析以外，另一种主要方法是对算法的效率做实验和经验分析。



(a)



(b)

关于本讲内容



祝各位学习愉快!

感谢观看！

讲解人：陆伟